

Hiero Workflow App

Five-Page Architecture Plan

arc42-compliant · Microkernel pattern · Draft for explorer3 review · LFD T Mentorship 2026

Core Principle and Pattern

Core principle: build the GitHub App as the final product from day one. Centralise reusable automation *decision logic* in a shared kernel; preserve SDK repository autonomy over workflow triggers, permissions, and security controls.

Architecture pattern: Microkernel Architecture (Richards, 2015). The shared policy kernel resides in a set of pure policy modules; the GitHub App shell (listener → normalizer → router → dispatcher → executor → audit logger) and an optional GitHub Actions compatibility adapter are delivery plug-ins over the same kernel. **arc42 coverage:** Sections 1–12 addressed in this plan. C4 views included for L1 (Context), L2 (Container), and L3 (Component).

This document is intentionally architecture-first. A separate phased delivery and migration document derives from this architecture and is out of scope here.

1. Introduction and Goals

1.1. Top Goals (Quality Drivers)

- **Security:** one trust boundary at the webhook listener; no policy module receives an unverified payload; all tokens are installation-scoped.
- **Maintainability:** fix automation logic once in a shared policy module and adopt deliberately across all SDK repositories.
- **Evolvability:** the policy kernel is runtime-agnostic; the Probot App and the GitHub Actions compatibility adapter call the same module interfaces without duplication.
- **Testability:** policy modules are pure functions; prove parity via fixture tests before any write mode is enabled upstream.
- **Adoptability:** each SDK repository adopts one module at a time via a config flag; rollback requires only setting `enabled: false`.

1.2. Stakeholders and Expectations

Stakeholder	Expectation
SDK maintainers	Retain governance and policy control; approve rollout timing; retain security gates.
SDK repositories	Keep CI/CD workflows, runner config, checkout, harden-runner, and local secrets.
GitHub App	Receive webhooks; run shared policy modules; write back via installation token.
GitHub Actions adapter	Optional compatibility path for testing; never the primary architecture.
Alternative runtime adapters	Any future adapter reuses the same policy module interfaces without duplication.

2. Constraints

- The GitHub App is the primary runtime from day one; GitHub Actions is a compatibility/testing adapter only.
- SDK repositories retain security controls permanently: workflow YAML, triggers, permissions, harden-runner, checkout, concurrency, artifact handling, and secrets.
- All GitHub API mutations originate from the App’s executor using installation-scoped tokens derived from a secure vault; no policy module holds credentials.
- Repository policy config (`.github/hiero-automation.yml`) is owned and protected by each SDK and read by the App via the GitHub API; it is never written by the App.
- Cross-SDK behaviour differences require fixture-parity tests before any write mode is enabled in an upstream repository.
- The first production policy module is `/assign`; PR review-sync and lifecycle modules follow only after the App shell is proven.

3. Context and Scope

The architecture boundary is deliberate: SDK repositories remain the security and governance boundary; the Hiero Workflow App becomes the shared decision boundary for reusable automation logic, receiving GitHub webhook events and writing back through installation-scoped tokens.

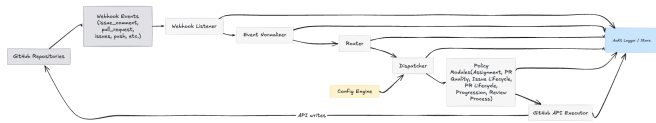


Fig.1: C4 L1 context / end-state architecture.

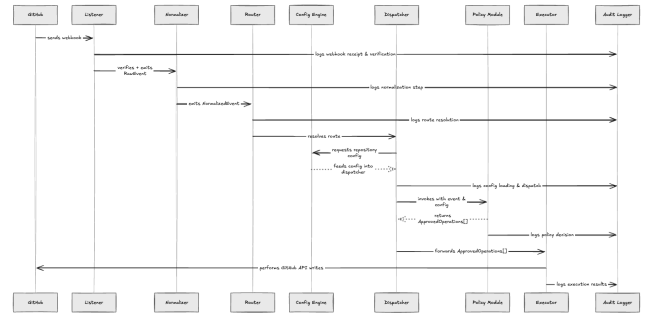


Fig.2: Component interaction / data-flow.

3.1. Scope

In Scope

- The eight-component GitHub App shell and all policy modules.
- Per-SDK config schema, validation, audit logging, and fixture parity tests.
- Incremental module adoption with config-flag rollback.

Out of Scope

Migrating SDK CI/CD workflows; hiding harden-runner or permissions; forcing C++ contributor-facing workflows onto an unproven wrapper; building lifecycle or review modules before the **/assign** slice proves the pipeline end-to-end.

4. Solution Strategy

Microkernel architecture: a small, pure policy kernel (no I/O) with a thin App shell for delivery. GitHub Actions is an optional compatibility adapter. Start with **/assign** as the minimal end-to-end slice to validate the shell before adding modules.

4.1. Microkernel Pattern Fit

Attribute	Expected fit	Rationale for this system
Agility	High	New automations register in the kernel without modifying the shell.
Testability	High	Policy modules are pure functions; tested in isolation with golden fixtures.
Deployability	High	Modules are versioned and enabled independently per SDK via config flag.
Scalability	Medium-Low	Workload bounded by SDK webhook events; not high-throughput service traffic.
Simplicity	Medium	Clear shell/module boundaries reduce duplication but require adapter discipline.

5. Building Block View

5.1. C4 L2 — Container View

Container	Responsibility and interaction
GitHub App service	Hosts all eight pipeline components; receives webhooks; writes via executor.
Repository config files	Per-SDK <code>.github/hiero-automation.yml</code> ; read by the Config Engine.
SDK repositories	Own CI/CD workflows, runner jobs, secrets; install the App to receive automation.
GitHub API	External system for comments, labels, assignments, and PR metadata.
GitHub Actions adapter	Optional compatibility container for canary testing and phased transition.

5.2. C4 L3 — Component View (Eight-Stage Pipeline)

Component	Responsibility
Webhook Listener	Receives HTTPS events; verifies signatures and rejects invalid deliveries.
Event Normalizer	Produces a stable <code>NormalizedEvent</code> for policy modules.
Router	Maps events/commands to product routes; routes register declaratively.
Config Engine	Loads and validates per-repo config; caches typed settings (TTL).
Dispatcher	Selects the module and forwards validated context and operations to the executor.
Policy Modules	Pure decision functions returning <code>ApprovedOperation[]</code> (no I/O).
GitHub API Executor	Executes operations with installation tokens; enforces idempotency and rate-limit handling.
Audit Logger / Store	Append-only structured audit per pipeline stage.

5.3. Three-Bucket Classification

Bucket	Examples	Rule
Policy kernel	Assignment decisions, PR check rules, lifecycle state machines, comment builders, label transition logic, idempotency checks, parity fixtures.	Lives in a policy module; built once; shared across all SDK installs.
Repo-local boundary	CI/CD workflow YAML, triggers, permissions, checkout, harden-runner, concurrency, artifact wiring, secrets, token choice.	Stays in each SDK repo permanently; never moved into the App.
Repo-specific policy	Label names, team handles, doc links, assignment limits, enabled modules, skill thresholds.	Expressed in the protected local config file; validated by the central schema.

6. Runtime View

6.1. GitHub App Runtime (Primary)

1. GitHub delivers a webhook to the App's HTTPS endpoint.
2. The Listener verifies the HMAC-SHA256 signature and installation scope; rejects on failure before any further processing.
3. The Normalizer converts the raw payload into a `NormalizedEvent`.
4. The Router resolves the event and any embedded command to a named product route.
5. The Config Engine loads and validates the target repository's policy config from the default branch via the installation token.
6. The Dispatcher selects the correct policy module and invokes it with the event context and validated config.
7. The policy module evaluates its guard conditions and returns `ApprovedOperation[]` (or a rejection reason).
8. The Executor performs approved GitHub API writes using an installation-scoped token. Every write is gated by an idempotency check and a delivery-ID dedup lookup.
9. The Audit Logger appends a complete decision record at each stage, including no-op events.

6.2. GitHub Actions Adapter (Compatibility Path)

1. An SDK workflow calls the shared action from the App repository by a pinned SHA.
2. The adapter validates GitHub Actions inputs and loads config from the repository via the checkout step (not via the App installation token).
3. The adapter calls the same policy module interface used by the App runtime.
4. This path is used for canary testing and phased transition only; it is not the production architecture and does not carry an audit log guarantee.

7. Deployment View

- The App runs as a containerised Node.js/TypeScript service (e.g., Cloud Run or Railway). The hosting runtime model must be confirmed as an ADR before Phase 1 begins, because it determines token cache, dedup store, and concurrency design.
- Private keys and secrets are stored in a secure vault (e.g., AWS Secrets Manager). Installation tokens are derived at runtime, cached for 55-minute TTLS, and refreshed with single-flight locks.
- The App exposes `/healthz` (liveness) and `/readyz` (readiness including GitHub API path, cache, and dedup store).

- The App repository must be org-owned with protected default branch, CODEOWNERS on config and schema files, named release owners, and documented incident/rollback ownership before any upstream SDK production install.
- Repository policy config is privileged: changes to `.github/hiero-automation.yml` require protected-branch review because they directly change automation behaviour.

Migration sequence implied by this architecture:

Phase 0	Architecture agreement; <code>/assign</code> confirmed as first slice; hosting ADR drafted.
Phase 1	Build App shell (listener → audit), no policy modules; all synthetic events route correctly.
Phase 2	<code>/assign</code> minimal in sandbox; fixture parity against Python and C++ golden cases.
Phase 3	<code>/assign</code> guards; dry-run pilot; midterm checkpoint by August 31.
Phase 4+	PR Quality module; Issue/PR Lifecycle; Progression; Review Process.

8. Crosscutting Concepts

- **Security:** one trust boundary at the listener; installation-scoped tokens; config loaded from the protected default branch via the API; strict Zod schema validation with fail-closed on unknown high-risk fields.
- **Configuration:** per-SDK policy config is a typed, versioned contract (JSON schema with 30-day backward-compatibility window and deprecation warnings). Missing config yields a virtual config where all modules are `enabled: false`.
- **Testing:** policy modules tested as pure functions; fixture/golden tests prove parity before write mode; 90%+ coverage target on all state-mutating logic.
- **Observability:** structured JSON logs with stable fields and a correlation ID propagated from listener to executor; audit record for every pipeline event; `events.received`, `events.failed`, `github.api.write.latency` and dedup/token/lock metrics exposed for alerting.
- **Reliability:** delivery-ID deduplication (24h window); per-entity concurrency locks (Redis or Postgres advisory); idempotency keys and bot-marker checks before every GitHub write; exponential back-off in the executor.
- **Versioning:** config schema versioned; modules declare compatible `schema_version` in their manifest; SDKs adopt modules independently and roll back by setting `enabled: false` in their config file.

9. Architectural Decisions

- **GitHub App as primary runtime, not GitHub Actions.** Actions is a compatibility/testing adapter. The App receives webhooks directly, owns the full event pipeline, and uses installation tokens. This is V2 architecture from day one, not a V1.5 wrapper evolved later.
- **Microkernel / plug-in policy modules.** Policy modules are pure functions registered in the dispatcher's module registry. Adding a new automation requires implementing the module interface and registering a route — zero changes to the listener, normalizer, or router.
- **`/assign` as the first product slice.** It forces the complete eight-stage pipeline to work end-to-end before any module-specific complexity is introduced. Review-sync and PR quality follow after the shell is proven.
- **Executor centralises all GitHub API writes.** No policy module holds credentials or performs I/O. This makes idempotency, audit logging, retry logic, and permission governance centrally enforceable.
- **Config as a typed policy surface, not code.** Repositories express governance intent through a schema-validated YAML file. They cannot inject arbitrary behaviour through config.
- **Audit logger as a first-class component.** Every event produces an audit record. Maintainer trust is earned through transparency, not promised through documentation.
- **Do not centralise SDK CI/CD workflow YAML.** Centralise only where duplication creates real maintenance cost. Security boundaries, runner configuration, and checkout strategy remain local permanently.

10. Quality Requirements

Quality goal	Scenario or test
Security	A forged webhook is rejected at the listener before any routing logic runs. A config file with unknown high-risk fields causes the affected module to fail closed without crashing any other module.
Maintainability	A bug fix in the Assignment Policy is deployed once and adopts across all installed SDK repositories by updating a single release reference.
Evolvability	Any alternative runtime adapter calls the same policy module interfaces as the primary GitHub App, with zero changes to any policy module.
Testability	Before write mode, fixture-parity tests reproduce the expected outcome for every golden case from existing Python and C++ behaviour, not only human dry-run log review.
Adoptability	Each SDK enables or disables any module independently via a config flag. Rollback takes effect within 5 minutes (config cache TTL).

Reliability	Concurrent webhooks targeting the same issue are serialised by per-entity concurrency locks. A replayed delivery ID is deduplicated within a 24h window with no duplicate write.
-------------	--

11. Risks and Technical Debt

Risk or debt	Mitigation or note
App shell defect impacts all SDK installs	Strict phase gating: no policy module ships until the shell passes security, schema, and routing tests. Module-level <code>enabled</code> flags limit blast radius.
Behaviour drift between Python and C++	Fixture-parity test suite required before any upstream write mode; per-SDK config allows divergent policy values without forking module code.
Hosting model deferred too long	ADR for runtime model (container vs. serverless) must close before Phase 1; deferral risks incompatible token cache and dedup store designs.
Config injection via malformed YAML	Zod schema rejects unknown high-risk fields; config loaded via installation token from the protected default branch only; arbitrary scripts in config are structurally impossible.
Installation token expiry mid-execution	55-minute cache TTL with single-flight refresh; circuit breaker on repeated refresh failures; events queued for manual reconciliation if circuit trips.
Premature teardown of legacy scripts	No legacy script deleted until its replacement module passes parity tests, a one-week controlled pilot, and explicit maintainer sign-off.
C++ contributor-facing automation constraints unknown	C++ workflows remain local; a behaviour-mapping investigation precedes any App module targeting C++ contributor-facing automations.

12. Glossary

Term	Meaning
Hiero Workflow App	The GitHub App that hosts the shared automation pipeline.
App shell	The eight-component pipeline: listener, normalizer, router, config engine, dispatcher, policy modules, executor, and audit logger.
Policy module	A pure decision function that receives a validated event context and config and returns <code>ApprovedOperation[]</code> with no I/O.
Config Engine	Component that loads, validates, and caches per-SDK policy config via the GitHub API.
Executor	Component that performs all GitHub API writes; enforces idempotency, rate limits, and audit.
Audit Logger	Append-only structured log capturing every pipeline stage decision.
NormalizedEvent	Stable internal event model; policy modules never depend on raw webhook payloads.
Delivery-ID dedup	Mechanism to reject replayed webhook deliveries within a 24h window.
Installation token	Short-lived GitHub credential scoped to one App installation; derived from a vault-stored private key.
Fixture / parity test	Test that verifies the App module reproduces the expected outcome of existing SDK behaviour.
GitHub Actions adapter	Optional compatibility path for canary testing; not the primary architecture.
harden-runner	StepSecurity action that hardens GitHub Actions runners; always stays visible in SDK workflows.